

TopoMem: Runtime-Topology-Guided Retrieval in the AIOps Agent Memory Loop

Renhong Zhang*, Hongxian Yang*, and Huabang Liu†

*University of Shanghai for Science and Technology, Shanghai, China
{2324010140, 2324010133}@st.usst.edu.cn

†Tencent Technology (Shenzhen) Company Limited, Shenzhen, China
huabangliu@tencent.com

Corresponding author: Renhong Zhang (2324010140@st.usst.edu.cn).

Abstract—Operational memory helps an incident-diagnosis agent only if the right evidence arrives before investigation. We identify the *multi-hop vocabulary gap*: symptom and root-cause records use service-scoped vocabularies, while their connecting edge exists only in the current runtime trace. TopoMem makes runtime topology a retrieval control signal through a trace-guided service walk and signature reranking, with multipath routing for conflicting traces. On an agent-generated corpus of 55 incidents and 548 operational documents, its trace walk achieves a root-document hit rate within the top five of 0.927, versus 0.636 for lexical retrieval and 0.218 for ApeRAG given the same path as text. The full method reaches 0.964 and improves mean reciprocal rank within the top five by 0.140. Multipath routing raises conflict-case hit rates from 0.001–0.053 to 0.919, while a controlled diagnostic study reduces the wrong-first-tool-target rate from 0.54 to 0.08. TopoMem turns operational-memory retrieval from passive similarity search into incident-grounded navigation.

Index Terms—AIOps, incident diagnosis, operational memory, multi-hop retrieval

I. INTRODUCTION

An enterprise AIOps agent operates as a recurring memory loop: *incident alert* → *trace/log collection* → *memory retrieval* → *evidence packet* → *agent investigation* → *validated write-back*. After resolution, the agent consolidates service profiles, runbooks, alert and change summaries, causal observations, failure-pattern notes, and remediation records. On the next alert, retrieval occurs before the agent forms its initial hypothesis and selects telemetry tools. A wrong first packet can therefore direct log, metric, and trace queries to the wrong service and consume a bounded investigation budget. Retrieval-augmented generation provides the interface between accumulated operational experience and agent reasoning [1].

The design targets three requirements. 1) **Structural retrieval**: cross service and vocabulary boundaries rather than rank documents independently. 2) **Incident grounding**: use current runtime evidence because a failure path may be absent from historical documents. 3) **Lightweight evolution**: accept learned records without large language model (LLM)-based graph reconstruction after every incident. Document-derived graph retrieval can encode relations extracted from indexed content [2], [3], but a graph built only from that content does not observe a runtime edge that appears for the current incident.

The problem. This first-step retrieval can fail for multi-hop incidents. Consider a frontend reporting HTTP 500 errors while the root cause is a three-hop PostgreSQL connection-pool failure documented only with database terms. No symptom term need appear in the root record. This is the classical *vocabulary gap* [4], but in microservice memory it can become structural when service-specific records use different terms. Text similarity alone has no direct representation of an anomalous call-chain edge that exists in telemetry rather than a stored document. With partial overlap, flat retrieval can still over-rank symptom and decoy records: in our evaluation corpus, BM25 and dense retrieval miss the root document in 20 and 22 of 55 incidents, respectively. A graph derived only from indexed historical content cannot supply an incident-specific edge absent from that content. Zero co-occurrence is the motivating extreme; the evaluation corpus retains partial overlap and uses independently authored queries and documents.

Our approach. We present TopoMem (**Topology-guided operational Memory** retrieval), an augmentation layer that uses the incident’s runtime trace as a *navigation structure*: walk the anomalous call chain from the symptom service to the root-cause service, then match the incident’s error signature against the documents at that service. This separates topological navigation (*which* service is failing) from failure-mode disambiguation (*which* failure mode there), using a signal that document-only ranking does not directly observe. The trace walk supplies the main retrieval gain, signature reranking improves ordering within the localized service region, and multipath routing extends the mechanism to unreliable or conflicting topology.

Contributions:

- 1) **Problem contribution**: we locate the multi-hop vocabulary gap inside the complete agent memory loop and explain its structural origin in service-owned operational records.
- 2) **Method contribution**: we separate trace-walk navigation from signature reranking, using incident topology to select a service region and runtime terms to prioritize a failure-mode record within that region (§III).
- 3) **Evidence contribution**: on a 55-incident, 548-document corpus, we report matched-telemetry baselines, including a full document-graph baseline, incident-level con-

fidence intervals, measured retrieval latency, signature sensitivity, stochastic trace-observation faults, a held-out evaluation of multipath routing, and a downstream diagnostic evaluation with a fixed agent configuration (§IV).

II. BACKGROUND AND PROBLEM

A. Operational Memory in an AIOps Agent

In our setting, an AIOps agent’s memory combines four sources: (1) human-authored runbooks, standard operating procedures (SOPs), application programming interface (API) contracts, and service profiles; (2) automatically ingested alert histories, deployment changes, and anomaly summaries; (3) validated incident, causal-chain, failure-pattern, and remediation records; and (4) a static dependency graph extracted from historical traces. At alert time, memory retrieval is the synchronous bridge between telemetry collection and reasoning: alert metadata, trace spans, and log fields form the incident query, and the returned evidence packet seeds the first hypothesis and service-targeted tool calls. After resolution, validated findings are appended as new memory for later incidents. Memory therefore carries operational experience across otherwise independent investigations; without useful recall, each alert begins again from raw telemetry, while a misleading packet can spend the tool budget on the wrong service. Retrieval latency is equally important because it lies on the critical path to the first diagnostic action rather than in an offline preparation stage.

B. The Multi-Hop Vocabulary Gap

Structural origin. When documents are maintained by service-specific teams, their vocabularies can diverge. A database team may maintain PostgreSQL runbooks in database vocabulary (`statement_timeout`, `autovacuum`, `connection pool exhaustion`), whereas a frontend team may maintain checkout guides in user-facing vocabulary (`HTTP 500`, `page latency`, `cart timeout`). Zero overlap is the extreme case; partial overlap can still favor a symptom or decoy document over the root-cause record, motivating both flat and topology-conditioned retrieval baselines.

Prevalence in public incident reports. To check whether this extreme is confined to our evaluation corpus, we analyzed 99 valid symptom–root section pairs collected from five public postmortem indexes [5]–[9]. High-IDF vocabulary was defined as the top 70% of tokens by inverse document frequency within the sample. Of these pairs, 49.5% shared no high-IDF token between the symptom and root-cause sections, and the median overlap was one token. This result shows that near-zero cross-section vocabulary overlap also occurs in independently published incident reports.

The consequence is the **multi-hop vocabulary gap**, illustrated in Fig. 1: the symptom query shares vocabulary only with symptom-layer documents, while root-cause documents at the far end of the call chain use entirely disjoint terminology. Text similarity has no explicit representation of the missing

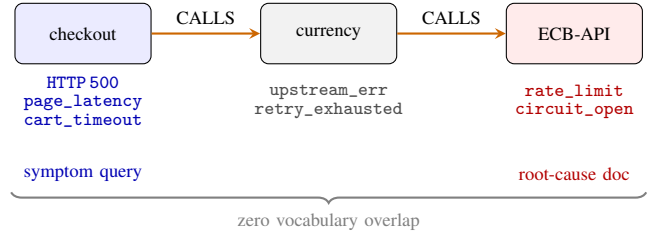


Fig. 1. Multi-hop vocabulary gap: symptom query and root-cause document share zero vocabulary. In this example, the multi-hop call path is observed in runtime graph G_t but is not encoded in the stored documents.

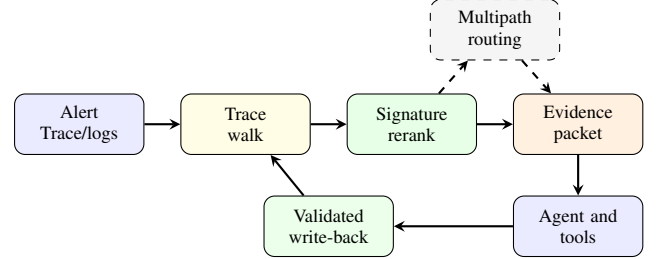


Fig. 2. Agent memory loop. The trace walk supplies topology navigation, the signature rerank refines ranking, and dashed arrows denote optional multipath routing (TopoMem-MP) under conflicting trace observations.

call-chain edge and must infer the relation indirectly, whereas the runtime trace exposes it for the current incident.

Why document-only graphs are insufficient here. A graph built only from stored content cannot index a call-chain edge absent from that content. TopoMem instead uses the incident’s runtime trace directly as the navigation structure. We evaluate the document-graph baseline with the same path and signature serialized as query text; unlike TopoMem, it does not execute trace traversal.

III. TOPOMEM DESIGN

TopoMem is a topology-aware augmentation layer over the memory store’s lexical and dense retrieval channels. Its two-stage mechanism uses the runtime trace graph to navigate to a candidate service region and the error signature to disambiguate failure modes. Fig. 2 places this mechanism inside the larger memory loop. We refer to the walk stage alone as TopoMem-Walk, to the full two-stage configuration as TopoMem, and to the multipath routing extension for unreliable traces as TopoMem-MP.

A. Stage 1: Trace-Guided Walk

The walk takes the symptom query q , the *symptom service* s_{symp} obtained from alert metadata, and the incident’s trace graph G_t . We perform breadth-first traversal from s_{symp} following anomalous CALLS edges, prioritized by cumulative error rate. An edge (u, v) is anomalous if its error rate exceeds a threshold (default 0.10). The walk terminates at a leaf in the anomalous subgraph or after maximum depth (default 5). All documents attached to visited services become candidates. This stage localizes the root-cause service without any text matching.

Input contract and fallback. With an incomplete trace, the walk traverses the observed prefix; with no trace, topology scores are disabled and ranking reduces to the lexical and dense channels (query-only fallback). Comparing multiple candidate terminals under conflicting observations is handled by TopoMem-MP (§III-C).

B. Stage 2: Signature Rerank

The root-cause service node may have many documents (runbooks for different failure modes, postmortems, service profiles). Stage 1 localizes the node; Stage 2 selects the right failure mode. We define an error signature $\sigma = \{t_1, t_2, \dots\}$ extracted from runtime evidence (log lines, trace span tags, alarm tokens) using a rule-based, non-LLM extractor: it reads log fields, trace span tags, and alarms to collect error keywords, HTTP/gRPC status codes, and error-pattern tokens. Documents at the root node are ranked by the signature score in Eq. 1:

$$\text{score}_{\text{sig}}(d_i, \sigma) = \frac{1}{|\sigma|} \sum_{t_j \in \sigma} \mathbf{1}[t_j \text{ occurs in } d_i] \quad (1)$$

with dense embedding similarity as a tiebreaker.

C. Multipath Routing (TopoMem-MP)

Greedy traversal can select the wrong terminal when a competing branch carries a high error rate. TopoMem-MP enumerates up to three anomalous paths. With one path it applies the two-stage ranking; with multiple paths it orders terminals by observed-signature support. A uniquely supported terminal uses its two-stage ranking, tied positive supports are fused, and zero support or no path falls back to BM25+signature (lexical fallback). Routing uses only incident-visible path and signature features; document role and root labels are reserved for evaluation.

IV. EVALUATION

We ask three questions. **RQ1:** under matched incident-visible telemetry, does trace-walk navigation improve root-document retrieval over flat and document-graph ranking, and does signature reranking further improve prioritization? **RQ2:** under stochastic trace-observation faults, where does the two-stage mechanism fail, and can multipath routing reduce that failure without clean-trace regression? **RQ3:** how does retrieval quality affect downstream incident diagnosis under a fixed agent configuration?

A. Experimental Setup

Agent-generated evaluation corpus. Operational documents can contain sensitive topology, configurations, and incident details. We therefore evaluate on an agent-generated corpus derived from representative operational requirements rather than production artifacts; no production document, service identifier, configuration, credential, vulnerability record, or incident payload is included. In the production workflow that motivates this study, the repository is not a conventional human-curated knowledge base: after an incident, the agent

constructs memory documents for its own future retrieval from operational evidence and validated outcomes. We mirror that process by having agents generate the evaluation documents from scenario-level operational evidence. Separate agents author the recurrence queries without document access, and document-generating agents have no access to those queries. The corpus contains 55 independent 3–4-service incident paths and 548 documents (about 165k words) spanning runbooks, postmortems, service profiles, change records, on-call hand-offs, and agent-memory notes; repeated query variants are not counted as independent samples. Across 30,140 incident-document pairs, none contains an exact copy of its query; signature-term coverage is 29.9% overall and 38.2% in root records. No root record contains a complete signature, and queries and signatures omit terminal service names. All reported comparisons use this corpus. Three LLM annotators assigned labels without access to ranking outputs: they agreed unanimously on 36 role-label cases, while majority vote resolved service labels for 84 documents without metadata.

Baselines. We implement BM25 with Okapi scoring, use the open-source Qwen3-Embedding-0.6B model for dense retrieval, and configure ApeRAG as the document-graph baseline [10]–[12]. The embedding model uses its query prompt; documents are split at paragraph boundaries (1,800-character maximum), and document similarity is the maximum normalized chunk cosine. TopoMem-Walk ranks by downstream trace position then dense similarity. TopoMem inserts observed-signature coverage between those two keys. BM25+signature and dense+signature receive the same incident-visible signature terms as TopoMem, but not its supplied path. ApeRAG uses local nomic-embed-text, GPT-5.4-mini graph extraction, and vector, full-text, and graph channels without reranking. Its three inputs are alert only, the same path serialized as text, and path plus the same signature serialized as text; it receives the runtime terms but does not execute the topology walk. Every method uses the same corpus manifest and document hashes. RQ1 uses the matched incident-visible anomalous path and signature to isolate ranking quality; RQ2 evaluates path extraction under trace-observation stress.

Stochastic trace-observation stress. RQ2 removes the supplied path. Each latent graph contains its primary path and a competing fan-out branch whose services have documents in the same corpus. We sample call/error counts and automatically extract anomalous paths under head/tail sampling, whole-edge dropout, depth-biased span loss, five-call low traffic, retries, error-label noise, stale high-error structure, and concurrent anomalous branches. An 18-path development subset calibrates multipath routing; the remaining 37 paths are held out for evaluation. We use 20 fixed-seed observation realizations and incident-level bootstrap intervals.

Metrics. Root Hit@5 is the fraction of incidents with at least one root-cause document in the top five. MRR@5 is the mean reciprocal rank of the first such document, with zero assigned when none appears in the top five. Role Coverage@5 is the mean fraction of four predefined roles—surface, bridge, root cause, and remediation—represented in the top five.

TABLE I
ROOT-DOCUMENT RETRIEVAL ON 55 INCIDENTS AND 548 DOCUMENTS.

Method	Root Hit@5	95% CI	MRR@5
BM25	0.636	[0.509, 0.764]	0.222
Dense (Qwen3-0.6B)	0.600	[0.473, 0.727]	0.212
BM25 + signature	0.745	[0.618, 0.855]	0.296
Dense + signature	0.636	[0.509, 0.764]	0.258
ApeRAG: alert only	0.200	[0.109, 0.309]	0.043
ApeRAG: trace text	0.218	[0.109, 0.327]	0.048
ApeRAG: trace + signature text	0.309	[0.182, 0.436]	0.078
TopoMem-Walk (trace + dense)	0.927	[0.855, 0.982]	0.515
TopoMem (trace + signature + dense)	0.964	[0.909, 1.000]	0.655

For stochastic signature conditions, we average 20 fixed-seed perturbation realizations within each incident and compute paired 95% confidence intervals (CIs) from 10,000 incident-level bootstrap resamples. RQ3 metrics are defined in §IV-D.

B. RQ1: Matched-Telemetry Retrieval

TopoMem retrieves a root document for 53 of 55 incidents (Table I), versus 35 for BM25, 33 for dense retrieval, 41 for BM25+signature, and 51 for TopoMem-Walk. Against BM25, TopoMem improves Hit@5 by 0.327 [0.200, 0.455]. Against TopoMem-Walk, the paired Hit@5 difference is 0.036 (95% CI [−0.055, 0.127]), whereas the paired MRR@5 difference is 0.140 [0.062, 0.223]. Signature evidence therefore primarily moves root documents earlier after topology localization.

Providing the path as text raises ApeRAG from 0.200 to 0.218 Hit@5, and adding signature text reaches 0.309; executing the same path as a navigation constraint reaches 0.927.

Why semantic and document-graph retrieval underperform. ApeRAG’s result reflects a mismatch between document-derived semantic structure and incident-specific operational causality. Its vector and full-text channels favor symptom-layer records that share HTTP status codes, alert phrases, and generic failure language with the query. Its graph channel can traverse only entities and relations extracted from stored documents; serializing a runtime path as text does not make the anomalous service edge a retrieval constraint. Dense embeddings soften lexical mismatch, but semantically similar failures in neighboring services remain strong decoys. Semantic retrieval is therefore useful within a topology-localized service region, as reflected by TopoMem’s dense tiebreaker, but is unreliable as the sole routing signal for cross-service multi-hop incidents.

On an Apple M4 Mac mini (16 GB), the p50/p95/p99 retrieval latencies are 164/248/357 ms for TopoMem-Walk and 167/251/360 ms for TopoMem, including CPU query embedding; the one-time 4.9 s model load is excluded. Under the reported ApeRAG configuration, index construction took 4144.8 s and trace-text retrieval required 3793/6945/9200 ms at p50/p95/p99; trace plus signature required 3907 ms at p50. Retrieval is a synchronous gate between alert ingestion and the agent’s first hypothesis, so its delay is added directly to time-to-first-action. Multi-second retrieval can also hold agent workers and amplify queues when several incidents arrive together. The measured sub-second path keeps memory

TABLE II
TOPOMEM SENSITIVITY TO SIGNATURE QUALITY (20 FIXED-SEED REALIZATIONS PER STOCHASTIC CONDITION).

Signature condition	Root Hit@5	MRR@5	Role Coverage@5
Observed signature	0.964	0.655	0.741
Retain 50%	0.965	0.609	0.729
50% + equal-length foreign noise	0.966	0.600	0.720
Query only (TopoMem-Walk)	0.927	0.515	0.727

TABLE III
HELD-OUT ROOT HIT@5 UNDER STOCHASTIC TRACE-OBSERVATION STRESS (37 PATHS, 548 DOCUMENTS, 20 FIXED-SEED REALIZATIONS).

Trace observation	BM25+sig.	TopoMem	TopoMem-MP
Clean finite-count trace	0.730	0.946	0.946
Head sampling 20%	0.730	0.926	0.928
Edge dropout 10%	0.730	0.853	0.878
Edge dropout 30%	0.730	0.685	0.773
Five-call low traffic	0.730	0.854	0.900
Stale high-error branch	0.730	0.001	0.919
Concurrent fault branch	0.730	0.053	0.919

access inside the online diagnostic loop. These values exclude telemetry collection and the remaining agent execution time.

Signature robustness. With a correct trace, signature dropout and equal-length foreign noise barely change Root Hit@5 (Table II) because topology dominates candidate selection. The MRR advantage over TopoMem-Walk remains positive with 50% retention (+0.094 [0.040, 0.150]) and with equal-length foreign noise (+0.085 [0.030, 0.142]). Thus, imperfect signature terms change ordering more than candidate localization when the trace is correct.

C. RQ2: Trace Observation and Multipath Routing

Automatic clean-path extraction reproduces held-out TopoMem Hit@5 0.946 (Table III). Whole-edge loss causes gradual degradation; at 30% dropout, TopoMem-MP improves over TopoMem by 0.088 [0.016, 0.161]. Low-traffic Hit@5 improves by 0.046 [0.022, 0.072]. When greedy traversal follows a competing high-error branch, TopoMem-MP improves stale and concurrent-branch Hit@5 by 0.918 [0.811, 0.999] and 0.866 [0.772, 0.942], respectively. Clean Hit@5 is unchanged. Small losses under error-biased tail sampling (−0.016 [−0.049, 0.000]), retry inflation (−0.014 [−0.041, 0.000]), and 5% label noise (−0.011 [−0.032, 0.000]) expose residual false fallback.

TopoMem-MP remains signature-dependent but degrades predictably. With 50% signature retention, stale/concurrent Hit@5 is 0.822; with equal-length foreign noise it is 0.707. With no signature, TopoMem-MP falls back to query-only BM25 at 0.595 rather than following the wrong branch.

D. RQ3: Controlled Downstream Diagnostic Study

RQ3 measures how the first evidence packet affects diagnosis by a bounded agent in a controlled study. The evaluation uses 24 multi-hop incidents—six failure families, each with four failure-mode variants—over a 312-document operational

TABLE IV
DOWNSTREAM DIAGNOSIS UNDER A FIXED AGENT CONFIGURATION (24 INCIDENTS). ROOT AND MODE SCORES RANGE FROM 0 TO 2; GOLD DOCS@5 IS A COUNT, AND THE FINAL TWO ROWS ARE RATES.

Metric	No context	BM25	Dense	TopoMem
Gold docs@5	0.00	0.04	0.00	2.33
Initial root	0.12	0.17	0.00	0.58
Final root	2.00	1.83	1.88	2.00
Final mode	1.75	1.33	1.67	2.00
Wrong first target	0.46	0.54	0.38	0.08
Root in ≤ 2 calls	0.67	0.46	0.83	1.00

store that includes decoy and neighboring-scenario records. Four retrieval conditions supply the packet: no context, BM25 top-5, dense top-5, and TopoMem top-5, inserted verbatim into the prompt. All other inputs are identical across conditions: the agent is GLM-5.1 at temperature 0.2 with a fixed system prompt; it must state an initial root-cause hypothesis before any tool call; it then investigates incident telemetry with at most three calls to `get_logs`, `get_metrics`, and `query_service` over at most two distinct services; it finally emits a root-cause service, failure mode, and remediation. Gold docs@5 counts gold documents in the evidence packet. Root-service scoring assigns 2 for an exact match, 1 when the gold service is mentioned, and 0 otherwise; failure-mode and remediation scores use token overlap with gold labels (0–2). Process metrics are computed from the recorded tool calls.

TopoMem packets contain a mean of 2.33 gold documents in the top five, versus 0.04 for BM25 and 0.00 for dense retrieval on this store (Table IV). The retrieval effect concentrates early in the loop: the initial root-cause score rises to 0.58 (BM25 0.17, dense 0.00), the wrong-first-tool-target rate falls to 0.08 (BM25 0.54, dense 0.38, no context 0.46), and the agent reaches the root service within its first two calls in every incident (BM25 0.46, dense 0.83). All conditions consume nearly the full budget (2.9–3.0 calls), so these differences reflect where calls are directed, not how many are made. Final root-cause scores converge (1.83–2.00) because three telemetry calls recover many incidents, while final failure-mode scores still separate (TopoMem 2.00 vs. BM25 1.33). Notably, BM25 packets underperform the no-context condition on wrong-first-target and root-reached rates: a misleading first packet is worse than none.

V. RELATED WORK

LLM-based incident diagnosis. LLMs have been applied to cloud-incident root cause analysis (RCA), tool-augmented workflows, and SOP-guided orchestration [13]–[16]. A systematic survey characterizes anomaly detection and RCA across service-based cloud applications [17]. TopoMem focuses on the upstream retrieval step that determines agent context.

Telemetry-based RCA. MicroRCA and Sleuth localize root causes from trace-derived observations, whereas CausalRCA applies causal inference to metric data [18]–[20]. TopoMem

instead uses runtime topology to retrieve explanatory operational records.

Multi-hop retrieval, vocabulary gap, and graph RAG. Work on the lexical chasm formalizes vocabulary mismatch in information retrieval (IR). Generative and hierarchical methods target multi-hop retrieval over semantically distant evidence, while GraphRAG systems use graph structure to expand or organize retrieval [21]–[24]. LightRAG and KG2RAG are document-derived instances; TopoMem’s distinction is the graph source: indexed-document relations versus incident-specific runtime topology.

Operational memory for agents. MemoryBank and A-Mem implement explicit long-term memory; Saha and Hoi mine root-cause knowledge from incident investigations; and FaultProFIT builds hierarchical incident-ticket profiles [25]–[28].

VI. LIMITATIONS AND FUTURE WORK

Three boundaries remain.

Agent-generated corpus. The evaluation corpus follows the target workflow in which an agent generates its own future memory from operational evidence and validated outcomes. This design supports controlled mechanism comparisons, while the public-postmortem analysis provides external context for the vocabulary gap. Evaluation on additional operational memory stores will test transfer across organizations.

Matched-input retrieval analysis (RQ1). RQ1 holds the incident-visible path and signature constant across topology-aware retrieval and matched-telemetry baselines to isolate ranking effects. RQ2 separately evaluates path extraction under degraded observation, distinguishing retrieval quality from telemetry acquisition.

Controlled trace stress (RQ2). RQ2 uses replayable parameterized observations spanning sampling loss, edge dropout, low traffic, retries, stale structure, and concurrent branches. This design isolates trace-failure mechanisms before production replay. Within this scope, the topology walk supplies the main Hit@5 gain, while signature reranking contributes earlier root-document ordering (MRR@5 +0.140).

VII. CONCLUSION

The AIOps agent memory loop depends on retrieving useful experience before a bounded investigation begins. We identified the multi-hop vocabulary gap as a structural mismatch between service-scoped records and incident-specific topology, and presented TopoMem, which conditions operational-document retrieval on incident-time topology. On 55 incidents and 548 documents, the trace-guided walk reaches Root Hit@5 0.927 versus 0.636 for BM25 and 0.218 for ApeRAG receiving the same path as text; signature reranking raises MRR@5 by 0.140, and multipath routing preserves clean Hit@5 while raising stale and concurrent-branch Hit@5 to 0.919. Under a fixed agent configuration, TopoMem packets improve initial hypotheses and reduce the wrong-first-tool-target rate from 0.54 to 0.08. TopoMem demonstrates that incident-time topology can guide operational-document retrieval across service

vocabulary boundaries: long-term documents store experience, while runtime topology supplies the temporary navigation structure needed to retrieve it. Future work will evaluate retrieval and downstream diagnosis in production incident replays.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [2] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, “LightRAG: Simple and fast retrieval-augmented generation,” in *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, 2025, pp. 10 746–10 761.
- [3] X. Zhu, Y. Xie, Y. Liu, Y. Li, and W. Hu, “Knowledge graph-guided retrieval augmented generation,” in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, 2025, pp. 8912–8924.
- [4] G. W. Furnas, T. K. Landauer, L. M. Gomez, and S. T. Dumais, “The vocabulary problem in human-system communication,” *Communications of the ACM*, vol. 30, no. 11, pp. 964–971, 1987.
- [5] D. Luu, “Post-mortems: A collection of incident reports,” GitHub repository, accessed: 2026-06-26. [Online]. Available: <https://github.com/danluu/post-mortems>
- [6] Dastergon, “Awesome site reliability engineering,” GitHub repository, accessed: 2026-06-26. [Online]. Available: <https://github.com/dastergon/awesome-sre>
- [7] R. Scott, “Awesome tech postmortems,” GitHub repository, accessed: 2026-06-26. [Online]. Available: <https://github.com/snakescott/awesome-tech-postmortems>
- [8] J. Lin, “Awesome postmortems,” GitHub repository, accessed: 2026-06-26. [Online]. Available: <https://github.com/jimmyl02/awesome-postmortems>
- [9] Saystone, “Awesome postmortem,” GitHub repository, accessed: 2026-06-26. [Online]. Available: <https://github.com/saystone/awesome-postmortem>
- [10] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [11] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin, F. Huang, and J. Zhou, “Qwen3 Embedding: Advancing text embedding and reranking through foundation models,” *arXiv preprint arXiv:2506.05176*, 2025.
- [12] ApeCloud, “ApeRAG,” <https://github.com/apecloud/ApeRAG>, 2026, accessed 2026-07-11.
- [13] Y. Chen, H. Xie, M. Ma, Y. Kang, X. Gao, L. Shi, Y. Cao, X. Gao, H. Fan, M. Wen, J. Zeng, S. Ghosh, X. Zhang, C. Zhang, Q. Lin, S. Rajmohan, D. Zhang, and T. Xu, “Automatic root cause analysis via large language models for cloud incidents,” in *Proceedings of the Nineteenth European Conference on Computer Systems*, 2024, pp. 674–688.
- [14] Z. Wang, Z. Liu, Y. Zhang, A. Zhong, J. Wang, F. Yin, L. Fan, L. Wu, and Q. Wen, “RCAgent: Cloud root cause analysis by autonomous agents with tool-augmented large language models,” in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, 2024, pp. 4966–4974.
- [15] X. Zhang, Q. Wang, M. Li, Y. Yuan, M. Xiao, F. Zhuang, and D. Yu, “TAMO: Fine-grained root cause analysis via tool-assisted LLM agent with multi-modality observation data in cloud-native systems,” *IEEE Transactions on Services Computing*, vol. 18, no. 6, pp. 4221–4233, 2025.
- [16] C. Pei, Z. Wang, F. Liu, Z. Li, Y. Liu, X. He, R. Kang, T. Zhang, J. Chen, J. Li, G. Xie, and D. Pei, “Flow-of-action: SOP enhanced LLM-based multi-agent system for root cause analysis,” in *Companion Proceedings of the ACM on Web Conference 2025*, 2025, pp. 422–431.
- [17] J. Soldani and A. Brogi, “Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: A survey,” *ACM Computing Surveys*, vol. 55, no. 3, pp. 59:1–59:39, 2023.
- [18] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, “MicroRCA: Root cause localization of performance issues in microservices,” in *NOMS 2020 – IEEE/IFIP Network Operations and Management Symposium*, 2020, pp. 1–9.
- [19] Y. Gan, G. Liu, X. Zhang, Q. Zhou, J. Wu, and J. Jiang, “Sleuth: A trace-based root cause analysis system for large-scale microservices with graph neural networks,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, 2023, pp. 324–337.
- [20] R. Xin, P. Chen, and Z. Zhao, “CausalRCA: Causal inference based precise fine-grained root cause localization for microservice applications,” *Journal of Systems and Software*, vol. 203, p. 111724, 2023.
- [21] A. L. Berger, R. Caruana, D. Cohn, D. Freitag, and V. O. Mittal, “Bridging the lexical chasm: Statistical approaches to answer-finding,” in *Proceedings of the 23rd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2000, pp. 192–199.
- [22] H. Lee, S. Yang, H. Oh, and M. Seo, “Generative multi-hop retrieval,” in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 2022, pp. 1417–1436.
- [23] A. Ghassel, I. Robinson, G. Tanase, H. Cooper, B. Thompson, Z. Han, V. N. Ioannidis, S. Adeshina, and H. Rangwala, “Hierarchical lexical graph for enhanced multi-hop retrieval,” in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, 2025, pp. 4457–4466.
- [24] B. Peng, Y. Zhu, Y. Liu, X. Bo, H. Shi, C. Hong, Y. Zhang, and S. Tang, “Graph retrieval-augmented generation: A survey,” *ACM Transactions on Information Systems*, vol. 44, no. 2, pp. 1–52, 2026.
- [25] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, “MemoryBank: Enhancing large language models with long-term memory,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 19 724–19 731.
- [26] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, “A-Mem: Agentic memory for LLM agents,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025, pp. 17 577–17 604.
- [27] A. Saha and S. C. H. Hoi, “Mining root cause knowledge from cloud service incident investigations for AIOps,” in *2022 IEEE/ACM 44th International Conference on Software Engineering: Software Engineering in Practice*, 2022, pp. 197–206.
- [28] J. Huang, J. Liu, Z. Chen, Z. Jiang, Y. Li, J. Gu, C. Feng, Z. Yang, Y. Yang, and M. R. Lyu, “FaultProfIT: Hierarchical fault profiling of incident tickets in large-scale cloud systems,” in *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*, 2024, pp. 392–404.